Frame（帧）
插件框架设计提案

RFC 001 / DESIGN PROPOSAL

面向第三方应用的插件装配、执行与生命周期管理

提交评审的设计

一个 Markdown 转换器可以出现在编辑器的「导出」菜单，也可以成为构建流水线的一步。两端接受同一份转换契约，插件就只需实现一次。安装来源、使用权限、选哪个实现，以及在哪里展示，都由各自的宿主决定。 本版沿用已评审提案的架构，名称统一为 Frame（帧）。API 代码为设计草图；可运行的入口以本仓库示例和开发指南为准。

> **框架承诺**
> Frame 将宿主批准的依赖绑定到具体实现，在一次使用期间保护它，并在原执行方确认退出后回收资源。

阅读路径

01 **理解边界 02–04**
事实归属、独立仓库和首版能力。

02 **体验 API 05–08**
编写转换插件与后台索引器，安装、选择、调用和停止它们。

03 **审查交付 09–13**
包信任、浏览器、故障恢复、发布与整体验收。

状态 设计基线 项目 Frame（帧） 日期 2026 年 11 月 1 日

01 / DOMAIN

一个插件接入宿主，谁做哪些决定

宿主定义领域语言，插件实现领域契约，Frame 对安装、精确绑定和受保护的使用期负责。

Unit 是独立装配与激活的单位，可有零到多个命名 import 和 export。Contract 描述值或服务的行为。Point 是宿主接收贡献的扩展点，关联元数据和可用值的契约。Event 为已准入的双方提供类型化事件。Use 保护一次实际访问。

事实	唯一能推进它的一方	其他组件可以做什么
服务的操作、数据、失败语义	发布领域契约的团队	生成类型、校验器与文档；接口同名不能替代契约身份。
某个 Point 怎么选、怎么排	消费方的业务规则	Frame 提供一致快照，再校验规则所选的候选。
一次选择究竟绑定到谁	Frame Host	驱动接收精确安装、release、Unit、export 和契约引用。
安装的选定版本与启用状态	Frame Host	存储记录已受理的转换；UI 和 CLI 读取其投影。
本次工作可读写哪些文档	拥有文档的宿主	通过绑定交付权限受限的 Reader 或 Writer。
资源是否已退出、业务是否完成	分别由原执行方、原业务方确认	Host 可收回保护；调用超时不能自行生成业务结论。

> **判断两份状态的办法**
> 数据库、Go 对象与 TypeScript DTO 都可以描述同一事实。若其中两方都能独立发起矛盾的状态变化，就应收回其中一条写入路径。

契约身份随影响行为的承诺改变。菜单位置、提供者地址、收集全部还是单选第一项，都不改变 Converter 的服务身份。具体选中哪个实现只保存在本次 Binding 中。

02 / REPOSITORY

独立仓库怎样组织

目标仓库为 github.com/Tangerg/frame。根目录承载实际 Go 模块，四个 npm 包围绕同一套公开契约协作。

目标目录 · 省略普通源文件

```
go.mod                  # github.com/Tangerg/frame
*.go                    # package frame: contracts, host, binding, lifetime
portable/  jsonvalue/  integrity/
distribution/  process/  runner/  sqlite/
control/                # control protocol and browser host adapter
contractgen/            # contract projections and author tooling
cmd/{frame,frame-runner}/
packages/{sdk,node,browser,react}/
examples/{buildcheck,document-indexer,browser-preview}/
protocol/{process,browser}/  # versioned wire schemas
conformance/            # wire fixtures and cross-language scenarios
docs/                   # author guide, host guide, protocol, releases
package.json            # private workspace root, single lockfile
```

Go 公共 API 保持短导入路径。只有一个独立发布的 Go 模块时，根 go.mod 有真实代码，不另设 go.work。Node、Browser 和 React 包按各自消费环境发布；SDK 提供共享类型与资源原语。协议生成工具的内部细节留在生成工具一侧。

依赖方向

Core 可以依赖稳定的值、摘要和契约校验能力。进程驱动、SQLite 和浏览器宿主适配器依赖 Core。Go 原生宿主的编译闭包可以不包含 SQLite、浏览器或 MCP SDK。Go 单模块的 go.mod 仍会列出可选适配包用到的依赖。

protocol/ 存放 Frame 自有线协议的机器可读定义，生成 Go/TS DTO；docs/ 解释行为和失败边界，conformance/ 检查实现。宿主的 documents 等领域契约由该领域包单独发布。

> **Flame 的位置**
> Flame 经已发布版本消费 Frame。外部 MCP 接入、工具映射、OAuth、模型回调和产品 UI 均留在 Flame；独立仓库的标准发行物和默认作者流程没有 MCP 依赖。

03 / CAPABILITIES

首版交付哪些能力

每项能力都有对应的调用者、公共承诺和可以独立验证的边界。

能力	主要使用者	首版承诺
声明与静态检查	所有插件作者	Unit 先声明依赖和导出。原生值沿用语言类型；跨进程值有公开描述、编解码和运行时校验。
扩展点、事件、观察	宿主与插件作者	一次快照覆盖多个 Point。Watch 的首份快照与后续修订衔接；队列溢出明确要求重新同步。事件投递依赖两端有效的 Use。
装配与生命周期	宿主作者	准备依赖、验证、激活和退休有明确时机。允许零导出后台 Unit 和运行期动态贡献。
执行环境	驱动与 SDK 作者	支持 Go 原生、同一浏览器 Realm 的 JS、通过 stdio 驱动的本地进程。静态资产按租约读取，可完全不启动进程。
归档与供应链	作者、分发工具、宿主	打包、离线检查、摘要校验、内容缓存与租约；宿主提供信任与权限策略。
耐久安装与恢复	需要重启恢复的宿主	可选 SQLite 实现；保留已批准的业务绑定及执行保护。内存宿主可以直接使用 Core。
作者工具与语言 SDK	Go、TS、其他语言作者	Go/Node 首批跨进程绑定；其他语言可按线协议和一致性向量实现。浏览器与 React API 单独导入。

新增 profile 时，驱动实现执行约束，归档校验器描述其所需入口。由宿主显式配对装配；标准归档校验仍统一负责路径、资源上限与摘要。这样新增执行环境不必修改 Core 的封闭枚举。

> **扩展尺度**
> v1 交付一种完整的本机跨进程方案。远程调度、插件市场、通用沙箱和特定业务的工作流可由宿主或专门组件提供。

04 / GO AUTHOR

写一个 Go 转换插件

领域包 documents 发布 Reader、Converter 与贡献点 Converters。作者只写自己的转换逻辑。

Reader 与 Converter 是编辑器和构建器共同接受的领域契约。Converters 的贡献元数据包括格式名。作者声明输入 slot 与输出 token 后，用这两个 token 完成绑定。下面是拟议 API 草图；文中的示例用于评审调用方式。

Go · 原生 Unit

```
var source = frame.Required("source", documents.Reader)
var markdown = frame.Offer("markdown", documents.Converters)

func Unit() (*frame.Unit, error) {
    return frame.DefineUnit(frame.UnitSpec{
        ID:      "markdown-converter",
        Imports: frame.Imports(source),
        Exports: frame.Exports(markdown),
        Prepare: func(s *frame.Setup) error {
            files, err := source.Get(s)
            if err != nil {
                return err
            }
            meta := documents.Format{Name: "markdown"}
            return markdown.Bind(s, meta, NewConverter(files))
        },
    })
}
```

DefineUnit 校验重名、基数和声明形状。Get 核对 slot 的原始契约；只有正确的可选 slot 才能报告缺席。Bind 仅暂存当前 Unit 的实现，通过最终准入后统一发布。动态贡献在运行期取得独立 publication 身份，随后可单独撤回。

作者体验的判据

slot 名、导出名和契约不必在 setup 再写一次。一个拼错的名字、缺席的可选依赖和类型不相符的实现，必须得到不同且可定位的结果。

05 / PROCESS AUTHOR

写一个没有导出的后台索引器

Node 插件只导入 Reader 和 IndexWriter。它可以在完全不使用 Agent 的编辑器中运行。

TypeScript · 独立进程入口，拟议 API

```
import { defineUnit } from "@frame/sdk";
import { serve } from "@frame/node";
import { reader, indexWriter } from "@acme/documents";

const indexer = defineUnit({
  id: "document-indexer",
  imports: { files: reader, index: indexWriter },
  exports: {},
  setup({ imports, onActivate }) {
    onActivate(work => {
      work.start(signal =>
        indexDocuments(imports.files, imports.index, signal));
    });
    return {};
  },
});

await serve(indexer);
```

indexDocuments 是插件自己的任务。work.start 先登记受管工作，再调度它；激活回调随即返回。停用时信号传给原任务，驱动等待其退出。SDK 把 imports 映射成有类型的代理，作者无需配置会话、回调地址或虚构一个对外服务。

Node 作者实现 portable 服务时，同一份领域描述生成客户端签名、服务端 handler 签名和运行时校验。浏览器作者也用 defineUnit，但同 Realm 的 UI 组件直接交付 JS 对象；声明过的 Point 在 setup 返回一次 contribution(metadata, value) 即可。

源码复用与发布物复用

两个宿主可运行同一进程归档，前提是都支持该平台、该进程 profile 与所声明的领域契约。Go 原生实现则随宿主编译。

06 / HOST

宿主如何选择、启动和使用插件

编辑器挑一个转换器，构建器收集全部转换器。两次决策可以不同，服务契约保持相同。

01 装配与安装

宿主显式注册领域契约、驱动和信任策略。导入已验证的归档后调用 Host.Install，得到安装记录，尚未运行插件。

02 授权与选择

宿主先判定当前文档的可见范围，取得对应的 grant，并把受限的领域服务交给绑定。Frame 根据同一快照核对所选成员和精确 provider。

Go · 使用获准的转换器，拟议 API

```
snapshot, err := host.Snapshot(ctx, documents.Converters)
if err != nil { return err }
pick, err := documents.Converters.SelectOne(snapshot, policy.Markdown)
if err != nil { return err }
plan, err := frame.Capture(snapshot, pick.Ref())
if err != nil { return err }

return host.With(ctx, grant, plan, func(a *frame.Access) error {
    member, err := pick.One(a)
    if err != nil { return err }
    return app.Export(ctx, member.Value, destination)
})
```

多个 Point 可从同一快照选择，Capture 一次接收所有选择。每组成员同时携带 Source、Metadata 和 Value，业务代码在同一受保护的回调内消费。原生语言可能留存引用，作者必须遵守借用期；跨进程句柄可由驱动验证期限与撤销。

无导出的入口如何运行

对上一页的索引器，宿主显式调用 host.StartUnit(ctx, grant, installed.Unit("document-indexer"))，拿到实例运行句柄。StartUnit 复用相同的绑定与准入路径。停止句柄针对该实例；禁用安装则封闭这次安装的新使用，并按约定退休受影响的实例。

07 / EXECUTION

本地进程协议怎样保持通用

v1 提供一条有界的双向 stdio 通道。Go 和 Node SDK 默认实现协议，其他语言可独立实现。

阶段	框架承诺	失败时的动作
预检	Host 核验 owner、scope、安装与绑定。需要耐久恢复时，先登记本次 attempt。	可预判的非法参数或无效授权，启动次数为零。
握手	协商协议版本和帧上限，核对已声明的导出及其契约。服务调用按 export、contract、member 定位。	拒绝缺失或伪造的导出，退休原尝试。
准备	把已冻结的 Unit 级 import slots 映射为有期限的句柄，包括明确的 optional 缺席。	驱动不能另选提供者；资源登记后才向插件返回。
激活	最终准入通过，才发布准备好的值并开始后台工作。显式 StartUnit 同样适用。	慢启动期间若撤销权限，拒发 Use，回收原实例。
调用	允许获准的反向调用、取消与有界流。执行 handler 时不占用接收锁。	区分未发送、远端已拒绝、结果未知；丢失响应不自动重放写入。
退休	停止新工作，跟踪已受理工作，由原执行方提交退出证明。	只续办原 attempt 和同一 ACK。退出未确认时继续保护其依赖。

会话内 request ID 只关联消息；业务命令 ID 由原业务 owner 管理。对端收到关闭请求、Runner 等到直接子进程退出、索引写入已提交，是三个独立事实。进程握手、排队和流式回调不需要认识 Tool 或模型。

同一插件服务多个文档时，句柄与原 scope、实例和期限绑定。仅在这些前提一致时复用已准备的绑定；若要复用底层进程，驱动仍须隔离不同授权的句柄和调用。

> **为什么选 stdio**
> 只有驱动接口，会迫使每个宿主重新解决双向调用和回收。采用 MCP 则让普通作者承担额外协议。一个具备一致性测试的本地参考实现，足以覆盖首版需求。

08 / DISTRIBUTION

从插件包到一次可信安装

发布物、缓存、安装、启用和执行各有独立状态。作者工具只推进自己拥有的那一步。

01 **作者构建并打包**

作者先编译目标程序。frame package 根据 manifest 收集指定文件；release 固定 Unit、契约引用、平台入口、资源清单及依赖。锁定信息从声明派生。

02 **宿主离线检查并决定信任**

frame inspect 可展示包、Unit、imports/exports 与请求的能力。verify 检查受限路径、文件与展开尺寸、摘要等。发行者签名和来源由宿主信任策略判断。检查插件包的过程不执行插件。

03 **内容入库与安装**

内容仓的 ImportArchive 导入已验证的不可变字节，Lease.Close 结束本次读取。Host.Install 推进安装记录；宿主可以先登记并保持禁用，之后显式启用。

04 **执行前授权**

宿主检查当前使用者、目标资源与本次请求。Host 只接受获准的精确绑定。纯资源包走字节租约，无须进程、调用会话或后台实例。

作者 **CLI** · 拟议入口

```
frame package --manifest plugin.json --out document-indexer.zip
frame inspect document-indexer.zip --format json
frame verify document-indexer.zip --policy publisher-policy.json
```

必须独立报告的三个判断

① 字节完整性是否通过。② 当前宿主能否解释并运行这个 profile。③ 当前使用者是否有权激活它。收到未知 profile 仍可检查通用归档；缺少受信形状校验器或执行器时，应明确拒绝执行。

本机进程的权限默认值

进程默认使用显式空环境，宿主只注入允许的变量。诊断脱敏，短期凭证不持久化。Reader 句柄可以限制到一个文档；原生二进制的文件和网络权限仍由 OS 与宿主部署策略约束。

09 / INTERACTION

浏览器、事件和释放共享同一套寿命语义

宿主负责菜单和布局。框架保护真实对象，使扩展撤回或视图卸载时不会误伤其他使用者。

浏览器 Runtime 在获准的 Realm 内执行模块，JS 对象保留在所属环境。Go Host 掌握安装、公共成员准入及使用资格。两端的协议消息投影同一批准结果；重连必须校验原凭据、代际与实际实例。

TypeScript · 两个独立挂载，拟议 API

```
const left = await mount(preview, { container: leftPane, props });
const right = await mount(preview, { container: rightPane, props });

await left.close();          // releases its own child use
await right.render(updated); // the sibling remains available
```

preview 是在父使用期内取得的类型化浏览器组件绑定。每次 mount 派生并持有自己的子 Use。卸载只释放该挂载，父使用期关闭则回收所有孩子，组件的真实 provider 始终得到保护。

操作	可观察承诺
Publish / Withdraw	发布成功后返回原 publication 句柄。撤回阻止新的获取，已有借用可以按原规则结束。
Watch / Snapshot	首份快照和变化流有一致起点。队列溢出提示重新同步；观察数据本身不授予执行权。
Event / Route	投递前核对发送方和接收方原 Use。事件负载有契约；关闭的一端不能接受新的投递。
BeginClose / Wait	关闭发起一次，可以多次观察或等待。取消本次等待不会再次执行 cleanup，也不会代替资源退出确认。

资源获取要把取得所有权、登记 cleanup 与失败回滚作为同一项承诺。借用的资源由原 owner 关闭；转移所有权或派生子 Use 应在调用入口可见。

10 / OUTCOMES

失败后，人和自动化应做出同一判断

UX、DX 与 AX 共享原操作事实。错误文本用于阅读，稳定类别及结构化证据用于编程。

观察到的结果	界面和脚本可采取的动作
预检拒绝，尚未执行	指出 unit、slot、期望契约与实际契约。更换声明、驱动或重新授权后再尝试。
新计划过期	尚未接受的计划可重新取快照。已落盘的业务记录必须恢复原绑定，找不到原实现时报错。
已提交，确认或刷新失败	保留 committed 结论与独立诊断。可继续读取或恢复原 operation，不能报成「安装失败」。
可能有副作用，结果未知	显示原 operation/attempt。若业务 owner 支持，查询原回执。没有回执不等于可以安全重发。
已退出，确认交接待完成	继续投递原确认。区分尚缺退出证明与已证明退出但尚未结清。

CLI / SDK · 预检失败的机器投影示意

```
{
  "stage": "bind", "code": "contract_mismatch",
  "unit": "document-indexer", "slot": "files",
  "expected": "documents.reader@2",
  "observed": "documents.reader@1",
  "execution": "not_started"
}
```

面向人的提示可以写成「索引器的 files 需要 Reader v2，当前提供 v1」。not_started 只在确定没有执行时输出。含副作用的操作不提供笼统的可重试布尔值。人类视图、SDK 错误和 CLI JSON 从原结论生成。

恢复入口属于原业务 owner

DTO 能传递数据，不能证明业务事务已接受绑定。普通调用方走新计划；恢复端口按持久记录身份读取已接受绑定，并验证原 release、配置和依赖。

11 / DELIVERY

从仓内模块到独立发布

允许 breaking 的开发期用于一次切换错误契约。正式对外后，包版本、线协议与领域契约分别维护。

交付批次	同批修改的内容	验收条件
A · **契约与权威**	区分服务和 Point 策略；收敛 slot/导出声明；隔离安装与业务 scope。	错误 token 明确报错；相同 epoch 的不同持久化 authority 无法误配。
B · **执行与消费方**	通用本机驱动、Go/Node 作者入口、Runner 恢复；迁移 Flame 受管插件及授权适配。	进程可零导出、反向调用与取消；外部 MCP 仍由 Flame 接入。
C · **仓库与发行物**	迁出源码和一致性套件，生成 Go 标签、可执行 ESM 与类型声明、平台 CLI/Runner。	干净项目能下载并运行；无需 Flame checkout、workspace 链接或本地候选代理。

独立 CI 的四个入口

① Go 以 GOWORK=off 从已标记版本构建外部宿主。② npm 用实际 tarball 验证 Node ESM、类型声明和承诺的 bundler 场景。③ 以发布的 CLI/Runner 在支持的平台运行真实子进程。④ Go 与 TS 运行同一组协议向量，并交叉调用。

线协议用明确版本握手；SDK 的 SemVer 描述作者 API 的兼容性；领域契约有自己的版本与行为身份。任何一项的版本号都不能代替另外两项的兼容检查。许可证、包命名空间和发布签名由仓库所有者在首次公开发布前确认。

> **迁移未完成的工作**
> 可再生的描述符和归档统一重建。旧任务和仍有副作用的 attempt 由原 owner 结清、显式退役或验证转换；切换 profile、改 digest 和删除开发数据不能产生退出或业务完成证明。

12 / REVIEW

怎样判定这份设计可以实施

让三类真实使用者独立完成任务，再用可观察的行为决定是否通过。

外部验收用例	通过标准
非 Agent 编辑器	作者仅凭发布文档交付零导出的 Node 索引器。授权一个文档后显式启动；预先无效的 grant 使进程启动与 OnActivate 次数均为零。
跨宿主复用	编辑器单选，构建器收集多个转换器。共享服务的契约摘要不变；同 profile 和平台可使用同一归档。
资源与自定义驱动	纯主题归档读取资产时创建零个子进程。受信第三方 profile 可打包、检查、安装并执行；未知 profile 不能擅自激活。
运行期撤回与 UI	撤回后新 Use 被拒，旧 Use 正常收尾。关闭一个挂载不影响兄弟挂载，父 Use 关闭后两者均退出。
恢复与自动化	响应丢失或 Host 在 ACK 交接前后崩溃，脚本能定位同一原操作；恢复不新增子进程，不重放未确认业务写入。
真正独立的发行物	空 Go 项目和 Node 项目经约定渠道安装并完成端到端示例。教程、作者 SDK 和标准协议无需 MCP 或 Flame。

本次评审请定下三项决定

① Frame 独立发布，MCP 产品适配归 Flame。② 采用一套 Unit/Contract/Point 模型，原生与进程路径共享绑定和使用期保证。③ 本机跨进程首版采用 stdio，首先提供 Go/Node 作者 SDK；所有发行物须通过外部消费验收。具体签名草图可随原型试用调整。

评估体验时记录首次插件运行耗时、需要重复手写的事实、定位契约错误所需步骤，以及故障后误重试造成的副作用。以同一任务比较改造前后，不用 API 数量替代体验。

设计来源 · 固定提交　AGENTS.md　WIT 的 imports / exports　go-plugin 的宿主接口